

```
In [1]: import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: import os
import time

from tqdm import tqdm

import numpy as np
import pandas as pd

import hyperopt.hp as hp

from sklearn.model_selection import train_test_split

from catboost import CatBoostClassifier

from sklift.datasets import fetch_x5
from sklift.models import ClassTransformation, TwoModels
from sklift.metrics import qini_auc_score, uplift_at_k

from causalml.inference.tree import UpliftTreeClassifier as UpliftTreeClassi

from upninja.dml.uplift_tree_dml import UpliftTreeRegressorDML
from upninja.tune.selection import UpliftTune

import matplotlib.pyplot as plt
import seaborn as sns
```

Failed to import duecredit due to No module named 'duecredit'

```
In [3]: SEED = 8
```

```
In [4]: %%time
```

```
dataset = fetch_x5()
dataset.data.keys()
```

CPU times: user 24.6 s, sys: 4.54 s, total: 29.2 s
Wall time: 31.4 s

```
Out[4]: dict_keys(['clients', 'train', 'purchases'])
```

```
In [5]: %%time
```

```
print(f'Dataset type: {type(dataset)}\n')
print(f'Dataset features shape: {dataset.data['clients'].shape}')
print(f'Dataset features shape: {dataset.data['train'].shape}')
print(f'Dataset target shape: {dataset.target.shape}')
print(f'Dataset treatment shape: {dataset.treatment.shape}')
```

Dataset type: <class 'sklearn.utils._bunch.Bunch'>

Dataset features shape: (400162, 5)

Dataset features shape: (200039, 1)

Dataset target shape: (200039,)

Dataset treatment shape: (200039,)

CPU times: user 81 µs, sys: 78 µs, total: 159 µs

Wall time: 164 µs

In [6]: `%%time`

```
# Извлечение данных
df_clients = dataset.data['clients'].set_index('client_id')
df_train = pd.concat([dataset.data['train'], dataset.treatment, dataset.target], axis=1)
indices_test = pd.Index(set(df_clients.index) - set(df_train.index))

# Извлечение признаков
df_features = df_clients.copy()
df_features['first_issue_time'] = \
    (pd.to_datetime(df_features['first_issue_date'])
     - pd.Timestamp('1970-01-01')) // pd.Timedelta('1s')
df_features['first_redeem_time'] = \
    (pd.to_datetime(df_features['first_redeem_date'])
     - pd.Timestamp('1970-01-01')) // pd.Timedelta('1s')
df_features['issue_redeem_delay'] = df_features['first_redeem_time'] \
    - df_features['first_issue_time']
df_features = df_features.drop(['first_issue_date', 'first_redeem_date'], axis=1)

indices_learn, indices_valid = train_test_split(df_train.index, test_size=0.2)
```

CPU times: user 152 ms, sys: 23.8 ms, total: 176 ms

Wall time: 176 ms

In [7]: `%%time`

```
X_train = df_features.loc[indices_learn, :]
y_train = df_train.loc[indices_learn, 'target']
treat_train = df_train.loc[indices_learn, 'treatment_flg']

X_val = df_features.loc[indices_valid, :]
y_val = df_train.loc[indices_valid, 'target']
treat_val = df_train.loc[indices_valid, 'treatment_flg']

X_train_full = df_features.loc[df_train.index, :]
y_train_full = df_train.loc[:, 'target']
treat_train_full = df_train.loc[:, 'treatment_flg']

X_test = df_features.loc[indices_test, :]

X_train['gender'] = X_train['gender'].map({'F': 0, 'U': -1, 'M': 1})
X_val['gender'] = X_val['gender'].map({'F': 0, 'U': -1, 'M': 1})
X_test['gender'] = X_test['gender'].map({'F': 0, 'U': -1, 'M': 1})

X_train.fillna(-1.0, inplace=True)
X_val.fillna(-1.0, inplace=True)
X_test.fillna(-1.0, inplace=True)
```

```
cat_features = ['gender']
```

CPU times: user 168 ms, sys: 4.77 ms, total: 173 ms

Wall time: 172 ms

✓ Train models

★ Class Transformation

In [8]: %%time

```
cb_params = cb_hp_space = {
    'iterations': hp.uniformint('iterations', 100, 2500),
    'depth': hp.uniformint('depth', 2, 10),
    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2),
    'l2_leaf_reg': hp.uniform('l2_leaf_reg', 1, 10),
    'verbose': False,
    'thread_count': -1,
    'random_state': SEED
}

cb_tune = UpliftTune(
    base_model_class=CatBoostClassifier,
    uplift_model_class=ClassTransformation,
    data=X_train,
    target=y_train,
    treatment=treat_train,
    space=cb_params,
    verbose=True,
    max_evals=10
)

t_time_start = time.process_time_ns()
cb_tuned = cb_tune.tune()['best_params']
t_time_end = time.process_time_ns()
```

100%|██████████| 10/10 [02:37<00:00, 15.79s/trial, best loss: -0.04954288228464684]

Optimization completed. Best score: 0.0495

Best parameters: {'depth': 2, 'iterations': 2166, 'l2_leaf_reg': 6.474242459540229, 'learning_rate': 0.06708005741019658, 'random_state': 8, 'thread_count': -1, 'verbose': False}

CPU times: user 21min 2s, sys: 2min 35s, total: 23min 38s

Wall time: 2min 37s

In [9]: %%time

```
ct = ClassTransformation(CatBoostClassifier(**cb_tuned))
f_time_start = time.process_time_ns()
ct = ct.fit(X_train, y_train, treat_train, estimator_fit_params={'cat_features': cat_features})
f_time_end = time.process_time_ns()
```

```
ct_uplift = ct.predict(X_val)
```

CPU times: user 3min, sys: 10.3 s, total: 3min 10s
Wall time: 21.6 s

★ DML Tree

```
In [10]: %%time

space = {
    'min_samples': hp.uniformint('min_samples', 20, 500),
    'max_depth': hp.uniformint('max_depth', 3, 20),
    'random_state': SEED
}

dml_tune = UpliftTune(
    uplift_model_class=UpliftTreeRegressorDML,
    data=X_train,
    target=y_train,
    treatment=treat_train,
    space=space,
    verbose=True,
    max_evals=10
)

t_time_start = time.process_time_ns()
dml_tuned = dml_tune.tune()['best_params']
t_time_end = time.process_time_ns()
```

100%|██████████| 10/10 [00:08<00:00, 1.12trial/s, best loss: -0.05889112202442145]

Optimization completed. Best score: 0.0589

Best parameters: {'max_depth': 14, 'min_samples': 411, 'random_state': 8}

CPU times: user 8.85 s, sys: 90.7 ms, total: 8.94 s

Wall time: 8.94 s

```
In [11]: %%time

dml = UpliftTreeRegressorDML(**dml_tuned)

f_time_start = time.process_time_ns()
dml = dml.fit(X_train, y_train, treat_train)
f_time_end = time.process_time_ns()

dml_uplift = dml.predict(X_val)
```

CPU times: user 550 ms, sys: 3.86 ms, total: 554 ms

Wall time: 553 ms

✓ Case

Оценим эффективность назначения коммуникаций при помощи uplift-моделей.

```

In [12]: def build_business_case_long(
    base_df,
    purchases_df,
    uplift_col,
    campaign_date=None,
    quantiles=(0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0),
    sms_cost=8
):
    """
    Бизнес-кейс:
    - клиенты ранжируются по uplift-score модели;
    - Top 10%, Top 20%, ..., Top 100% считаются накопительно сверху скоринга;
    - горизонты: 1, 2 и 3 недели;
    - target_horizon = факт покупки внутри горизонта;
    - ОП на клиента = средний чек ТГ - средний чек КГ - стоимость SMS;
    - Total increment = ОП на клиента * число клиентов в выбранной доле базы
    """

    base_df = base_df.copy()
    purchases_df = purchases_df.copy()

    purchases_df['transaction_datetime'] = pd.to_datetime(
        purchases_df['transaction_datetime']
    )

    if campaign_date is None:
        campaign_date = purchases_df['transaction_datetime'].min()
    else:
        campaign_date = pd.to_datetime(campaign_date)

    purchases_df['days_since_campaign'] = (
        purchases_df['transaction_datetime'] - campaign_date
    ).dt.days

    purchases_df = purchases_df[purchases_df['days_since_campaign'] >= 0].copy()

    horizons = {
        '1w': 7,
        '2w': 14,
        '3w': 21,
    }

    rows = []

    # Сортировка строго по убыванию uplift-score:
    # сначала самые топовые клиенты, потом хуже.
    base_sorted = (
        base_df
        .sort_values(by=uplift_col, ascending=False)
        .reset_index(drop=True)
    )

    total_n = len(base_sorted)

    for q in quantiles:

```

```

cutoff = int(total_n * q)
top_clients = base_sorted.iloc[:cutoff].copy()
top_client_ids = top_clients['client_id']

for horizon_name, max_days in horizons.items():

    horizon_purchases = purchases_df[
        (purchases_df['client_id'].isin(top_client_ids)) &
        (purchases_df['days_since_campaign'] <= max_days)
    ].copy()

    purchase_agg = (
        horizon_purchases
        .groupby('client_id')
        .agg(purchase_sum_horizon=('purchase_sum', 'sum'))
        .reset_index()
    )

    horizon_df = top_clients.merge(
        purchase_agg,
        on='client_id',
        how='left'
    )

    horizon_df['purchase_sum_horizon'] = (
        horizon_df['purchase_sum_horizon'].fillna(0)
    )

    horizon_df['target_horizon'] = (
        horizon_df['purchase_sum_horizon'] > 0
    ).astype(int)

    treat = horizon_df[horizon_df['target_horizon'] == 1]
    control = horizon_df[horizon_df['target_horizon'] == 0]

    conv_treat = treat['target_horizon'].mean()
    conv_control = control['target_horizon'].mean()
    uplift_conv = conv_treat - conv_control

    avg_check_treat = treat['purchase_sum_horizon'].mean()
    avg_check_control = control['purchase_sum_horizon'].mean()

    op_per_client = avg_check_treat - avg_check_control - sms_cost
    total_increment = op_per_client * len(horizon_df)

    rows.append({
        'Модель': uplift_col,
        'Горизонт': horizon_name,
        'Доля базы': f'Top {int(q * 100)}%',
        'Доля базы sort': int(q * 100),
        'Кол-во наблюдений': len(horizon_df),

        'Конверсия в КГ': conv_control,
        'Конверсия в ТГ': conv_treat,
        'Инкремент конверсии': uplift_conv,
    })

```

```

        'Средний чек в КГ': avg_check_control,
        'Средний чек в ТГ': avg_check_treat,

        'Инкремент ОП на клиента, руб': op_per_client,
        'Total increment, руб': total_increment,
    })

    return pd.DataFrame(rows)

```

```

In [13]: def make_burger_king_table(long_df):
    """
    Таблица как в Burger King:
    строки – Top N% по uplift-score;
    столбцы – блоки по горизонтам: 1 неделя, 2 недели, 3 недели.
    """

    long_df = long_df.copy()

    horizon_order = ['1w', '2w', '3w']

    horizon_titles = {
        '1w': '1 неделя',
        '2w': '2 неделя',
        '3w': '3 неделя',
    }

    long_df['conv_control_fmt'] = (
        (long_df['Конверсия в КГ'] * 100)
        .round(1)
        .astype(str)
        .str.replace('.', ',', regex=False)
        + '%'
    )

    long_df['conv_treat_fmt'] = (
        (long_df['Конверсия в ТГ'] * 100)
        .round(1)
        .astype(str)
        .str.replace('.', ',', regex=False)
        + '% '
        + long_df['Инкремент конверсии'].apply(
            lambda x: f"({x * 100:.1f}%) " if x >= 0 else f"({x * 100:.1f}%) "
        ).str.replace('.', ',', regex=False)
    )

    long_df['op_fmt'] = (
        long_df['Инкремент ОП на клиента, руб']
        .round(1)
    )

    long_df['total_increment_fmt'] = (
        long_df['Total increment, руб']
        .round(0)
        .astype(int)
    )

```

```

final = (
    long_df[['Доля базы', 'Доля базы sort', 'Кол-во наблюдений']]
    .drop_duplicates()
    .copy()
)

final = (
    final
    .sort_values('Доля базы sort')
    .drop(columns='Доля базы sort')
    .reset_index(drop=True)
)

for h in horizon_order:
    h_df = long_df[long_df['Горизонт'] == h].copy()

    h_df = h_df[[
        'Доля базы',
        'conv_control_fmt',
        'conv_treat_fmt',
        'op_fmt',
        'total_increment_fmt'
    ]].rename(columns={
        'conv_control_fmt': f'{horizon_titles[h]} | Конверсия в КГ',
        'conv_treat_fmt': f'{horizon_titles[h]} | Конверсия в ТГ',
        'op_fmt': f'{horizon_titles[h]} | Инкремент ОП на клиента, руб',
        'total_increment_fmt': f'{horizon_titles[h]} | Total increment,
    })

    final = final.merge(h_df, on='Доля базы', how='left')

return final

```

```

In [14]: X_val['ct_uplift'] = ct_uplift
X_val['dml_uplift'] = dml_uplift
X_val = pd.concat([X_val, treat_val, y_val], axis=1)

```

```

In [15]: X_val = X_val.merge(dataset.data['purchases'][['client_id', 'purchase_sum',

```

```

In [16]: dml_long = build_business_case_long(
    base_df=X_val,
    purchases_df=X_val,
    uplift_col='dml_uplift',
    campaign_date=None,
    quantiles=(0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0),
    sms_cost=5
)

dml_burger_king_table = make_burger_king_table(dml_long)

dml_burger_king_table

```

Out [16]:

	Доля базы	Кол-во наблюдений	1 неделя Конверсия в КГ	1 неделя Конверсия в ТГ	1 неделя Инкремент ОП на клиента, руб	1 неделя Total increment, руб	2 неделя Конверсия в КГ
0	Top 10%	685611	68,0%	63,9% (-4,1%)	-453.6	-310962685	78,1%
1	Top 20%	1371223	69,9%	67,2% (-2,7%)	-95.6	-131071036	80,6%
2	Top 30%	2056835	68,5%	66,8% (-1,8%)	188.0	386734438	79,5%
3	Top 40%	2742447	70,5%	68,9% (-1,6%)	529.0	1450676750	81,4%
4	Top 50%	3428059	72,1%	70,9% (-1,2%)	739.5	2535116951	83,0%
5	Top 60%	4113670	73,1%	72,1% (-1,0%)	526.8	2166965434	84,1%
6	Top 70%	4799282	72,6%	71,9% (-0,7%)	546.6	2623348640	83,3%
7	Top 80%	5484894	71,8%	71,3% (-0,6%)	272.6	1495389155	82,7%
8	Top 90%	6170506	71,5%	71,1% (-0,4%)	-3.1	-19398371	82,2%
9	Top 100%	6856118	70,3%	70,1% (-0,3%)	-180.2	-1235316451	81,2%

```
In [17]: ct_long = build_business_case_long(
    base_df=X_val,
    purchases_df=X_val,
    uplift_col='ct_uplift',
    campaign_date=None,
    quantiles=(0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0),
    sms_cost=5
)

ct_burger_king_table = make_burger_king_table(ct_long)

ct_burger_king_table
```

Out [17]:

	Доля базы	Кол-во наблюдений	1 неделя Конверсия в КГ	1 неделя Конверсия в ТГ	1 неделя Инкремент ОП на клиента, руб	1 неделя Total increment, руб	2 неделя Конверси: в КГ
0	Top 10%	685611	71,1%	64,8% (-6,3%)	-1501.3	-1029302395	82,4%
1	Top 20%	1371223	71,9%	67,7% (-4,2%)	-57.2	-78439355	83,3%
2	Top 30%	2056835	71,5%	70,3% (-1,3%)	665.4	1368515736	82,9%
3	Top 40%	2742447	72,0%	70,9% (-1,1%)	556.7	1526695310	83,2%
4	Top 50%	3428059	72,1%	71,4% (-0,7%)	424.6	1455425964	83,2%
5	Top 60%	4113670	72,3%	71,8% (-0,5%)	203.1	835546907	83,4%
6	Top 70%	4799282	72,7%	71,9% (-0,8%)	82.5	395985831	83,5%
7	Top 80%	5484894	72,4%	71,5% (-0,8%)	-29.6	-162446957	83,3%
8	Top 90%	6170506	72,1%	71,5% (-0,7%)	-123.8	-764004817	83,0%
9	Top 100%	6856118	70,3%	70,1% (-0,3%)	-180.2	-1235316451	81,2%